Complex Types

I know what you are thinking, we just got done with basic types, and that was a lot to take in, and now we are getting into complex types??? This is madness! However, complex types aren't as scary as they sound. So let's get into it!

Variables, as we know them so far, are great for storing exactly 1 thing, but let's be real, that's pretty basic. Luckily, there are types that are made for storing a bunch of things, and let us group them together treating them like one thing. We call these Collections. There are four main types of Collections in Swift.

- `Arrays` — Think of something like a list of items of the same `Type`.
- `Sets` — A `Set` is a list of items that contains only unique items with no particular order.
- `Tuple` — A `Tuple` is a collection of items that has a fixed size, unlike `Arrays` and `Sets` it can contain multiple different `Types` of data. It also allows you to name the various pieces of data it contains.
- `Dictionary` — A `Dictionary` is a collection of key, value pairs.

Don't worry, I know that was a bit of a vocabulary dump, but we are going to talk about each of these much more in depth, starting with `Arrays`.

Arrays

Creating an Empty Array

We are going to start by making an `Array` that is empty and doesn't yet store any values.

> **Note**
>
> Arrays and Lists are fundamentally different conceptually; however, in Swift specifically you will hear the names used interchageably.

`Arrays` are declared and referenced, typically using Square Brackets `[]`. So let's create our first empty `Array`!

```
// Has an error: `Empty collection literal requires and explicit type`
// Make sure to comment out this line after you see the error!
let emptyArray = []
```

Well, crap, we have an error. What does this error mean? “Empty collection literal requires an explicit type”. So remember what we talked about yesterday, how Swift is a strongly

typed language, but offers type inference when it can figure out what type you want something to be? This is an example of when there isn't enough information to infer what type the above `Array` is. Luckily, we have **Type Annotation** to save the day!

So if we are going to create an empty `Array` we have to specify what kind of values it would contain if it had any. There are two different ways that we can do this. The first is simply using **Type Annotation**; however, there is also another way we can create an empty `Array` that lets us specify a `Type` as well, so let's show both of those!

```
// Using Type Annotation
let emptyArrayOfStringOne: [String] = []
// Using the other form of empty Array creation
let emptyArrayOfStringTwo = [String]()
// There are other forms, but these are the important ones.
```

You can see that either way, we have to manually specify what `Type` is contained by the `Array`.

Creating an Array with Values

Another thing that you can do is create `Arrays` that have values in them, crazy, right?

```
let arrayOfArtists = ["NF", "Post Malone", "Johnny Cash"]
```

You will notice that this time an error isn't popping up, why might that be? It's because now there is enough information for Swift to infer what type the `Array` is storing, as we have a value that we put inside of it. In this case, it is an `[String]`.

Adding & Removing Elements From An Array

It's nice that `Arrays` can hold multiple elements, but that usefulness only goes so far. We need the ability to add and remove elements in order to get real power out of our `Arrays`. What steps do we need to take to do that? In most languages you would have to specify if an `Array` can have elements changed or now; however, Swift does that for us, simply by making our `Arrays` either constants or variables.

Let's start by making some variables to populate our list with, you can pass the values that a variable contains into an `Array` as long as the `Types` match!

```
let snorlax = "Snorlax"
let espeon = "Espeon"
let charizard = "Charizard"
let venusaur = "Venusaur"
```

```
var party: Array<String> = [snorlax, espeon, charizard, venusaur]
```

Oh boy, we have a new surprise! Another way that we can declare an Array!! Saying `Array<String>` is the same thing as saying `[String]`; however, it's recommended that you stick with the prior `[String]`. It is still important that you know both exist!

Now that we have created an `Array` with some values in it, let's add some more values to it!

```
party.append("Blastoise")
// party = [Snorlax, Espeon, Charizard, Venusaur, Blastoise]
```

More Syntax! Adding a `.` after our variable name popped up a bunch of different things. We will explain more about what's going on there later on. For now, let's talk about the thing we are using. `.append` is a function that appends, or adds, another element to our `Array`. Whenever we use the `append` function it's going to add whatever value we tell it to, to the end of our `Array` so now Blastoise is the last value contained in our `Array`.

What if instead of adding a new value to the end of our `Array` we wanted to put it somewhere specific like the second spot? There is another function made for this called insert, let's try using it!

```
party.insert("Pikachu", at: 1)
// party = [Snorlax, Pikachu, Espeon, Charizard, Venusaur, Blastoise]
```

You may be a little confused here, if we wanted to put the item in the second spot, why did we use the number one? And even more confusing, why did it work? This is a very important concept that you are going to have to learn to love, and that is that computers don't start counting at one, they start counting at 0.

We can also use these numbers to remove specific elements from an `Array`. Let's say I want to get rid of the third element of our `Array` in this case being `Espeon`. We can do that by using another function called `remove`.

```
party.remove(at: 2) // Third item is at an `Index` of 2
// party = [Snorlax, Pikachu, Charizard, Venusaur, Blastoise]
```

Retrieving Items From An Array

Being able to store values, but not read them seems pretty useless, right? So let's get into how we can read from our `Array`. When we are trying to get elements from our `Array` we need to specify which element we want to grab. The main way that we do that is by

referencing the `Index` of that element, aka the position that the element is at in the `Array`. So let's try to get the first element from our `Array` what `Index` do you think it is at? Your gut reaction may be that since it's the first element it would be at `Index` 1, but remember computers start at 0, so its actually at the 0 `Index`.

```
let firstPokemon = party[0]
// firstPokemon = "Snorlax"
```

We can also get the last element from an `Array` even without knowing how many elements are in it!

```
let lastPokemon = party.last
```

Bonus Array Functions

There are so many things you can do with `Arrays` and all the different collections we are going to cover today. Really quick, I want to go through a couple of cool ones rapid-fire style.

```
// Reverses and `Array` in place, only works on variables.
party.reverse()
// party = [Blastoise, Venusaur, Charizard, Pikachu, Snorlax]

// Creates a copy of the `Array` in reversed order.
let reversedPokemon = party.reversed()
// reversedPokemon = [Snorlax, Pikachu, Charizard, Venusaur, Blastoise]

// Gets the current number of items in an `Array`. Starts counting and 1
not 0
let pokemonCount = party.count
// pokemonCount = 5

// Lets say I wanted to use this new value to get the last element of an
Array.
// How could I do that?
let finalPokemon = party[party.count - 1]
// We subtract 1 to account for starting at 0 for indicies.
```

Sets

A `Set` is a collection of unordered unique items. Unique meaning there can be no duplicate items in this collection type. `Sets` unlike the other collections we have/are going

to cover today, don't get any special syntax to create them. Instead, they rely on the syntax we use to create `Arrays`, to an extent. Let's create a couple of sets!

```
let colors = Set(["Red", "Green", "Blue"])
let colorsDoubled = Set(["Red", "Red", "Green", "Green", "Blue",
"Blue"])

print(colors) // "Green", "Blue", "Red"
print(colorsDoubled) // "Blue", "Green", "Red"
```

As you can see in the above example, to create a `Set` we wrap the square brackets, like we see with `Arrays`, in parentheses prefixed by the word `Set`. You can also see that despite having two different input `Arrays` we ended up with nearly if not the same output. This is because each of the `Arrays` only contained 3 unique items. So the `Set` removed the duplicates. The reason they may be slightly different, is because `Sets` are also unordered, so the order we output them in may be different. However, the contents stay the same.

All the collections we are going to cover offer some shared functionality, so I won't go over everything they share. There are, however, differences between some of them, so I'm going to cover some differences, and leave it up to you to experiment and see the similarities.

Creating An Empty Set

In order for Swift to know what type our `Set` is, we need to specify what we want to store inside of it. We can do that by either providing elements, or specifying the type manually. We have shown how to do it with elements, so let's show how we can create empty `Sets`!

```
let emptySet = Set<Int>()
let emptySetTwo: Set<Double> = []
// There are others, like `Arrays` but these are the main ones.
```

Adding and Removing Elements From A Set

Adding and removing elements from a `Set` differs slightly from that of an `Array` this primarily comes down to the fact that `Sets` are unordered while `Arrays` are ordered. So things like accessing the `Index` of an item can't happen inside a `Set`.

```
var carBrands = Set(["Honda", "Toyota", "Ford"])
```

`Sets` don't have an `append` function as you can't add to the end of something that has no defined end, instead we rely on the `insert` function. This time `insert` is a little

different as we can't specify where we want to insert it, so we just simply say here is your new item.

```
carBrands.insert("Hyundai")
carBrands.insert("Hyundai")
// carBrands = Toyota, Hyundai, Ford, Honda
```

You can remove elements from a `Set`, using the remove function. This time instead of specifying a position, you specify an element you want removed.

```
carBrands.remove("Ford")
```

Dictionaries

A `Dictionary` stores associations of `Keys` and `Values` in an unordered collection. Each `Value` is associated with a unique `Key`. This `Key` acts as an identifier for that value within the `Dictionary`. Let's start by creating a dictionary of words, paired with their definitions, take the word dictionary very literally here. In this example, our word is our `Key` and the definition is our `Value`. When we create a `Dictionary`, we specify a `Key` then a `Value` separated by a `:` then comma separate those pairs.

```
let literalDictionary = [
    "Taradiddle": "A petty lie",
    "Firkin": "A small cask formerly used for liquids, butter, or fish",
    "Grok": "To understand profoundly and intuitively",
    "Erf": "A plot of land usually about half an acre in size"
]
```

`Dictionaries` are unique in comparison to the other collections we have covered so far, as they require multiple `Types`. Up until now all of our collections have stored a single `Type`, `Dictionaries` require two `Types`. One type for our `Keys` and one type for our `Values`. To show this off, let's create an empty `Dictionary` that would store numbers and their word equivalents.

```
var numbersAndNames = [Int: String]()
// or
let dictionaryCreation2: [Int: String] = [:] // [:] represents and
empty Dictionary
```

Adding And Removing Elements From A Dictionary

`Dictionaries` require both a `Key` and a `Value` in order to store anything, this makes the process a little different than `Set` and `Array`. First difference being there isn't an `append` or `insert` function. Instead, we access a key from a `Dictionary` and then set it to a new value. Let's add some values to the `numbersAndNames` variable we created.

```
numbersAndNames[1] = "One"
numbersAndNames[2] = "Two"
numbersAndNames[3] = "Three"

print(numbersAndNames)
// [2: "Two", 3: "Three", 1: "One"]
```

Removing `Values` should be a little more familiar, we use the `removeValue` function on our dictionary and we can remove anything. We just tell it which `Key` we want to remove!

```
numbersAndNames.removeValue(forKey: 1)
// [3: "Three", 2: "Two"]
```

Reading Values From A Dictionary

In order to read a `Value` from a `Dictionary` we need to know what `Key` to use. Let's create a new `Dictionary` that represents a student's attendance for a day.

```
let attendance = [
    "Ryan Reynolds": true,
    "Keanu Reeves": true,
    "Danny Devito": false,
    "Selena Gomez": true,
    "Emma Stone": true,
    "Betty White": false
]
```

Now, let's try and see if Ryan Reynolds showed up to class today.

```
let isRyanPresent = attendance["Ryan Reynolds"]
print(isRyanPresent) // WARNING: Expression implicitly coerced from
Bool? to Any
```

You will notice that when we try to print the value we get from our `Dictionary` we get a warning, and the print statement has this `Optional()` thing. An optional value is what is sounds like, it is a value that may or may not exist. If it doesn't exist, the value is `nil`. We

are going to talk a lot more about this concept later on. For now, just remember that if you try to get a value from a `Dictionary` it could be `nil`.

This is because when you get try to get a `Value` from a `Dictionary` by using its `Key` there may or may not be a `Value` associated with that `Key` in the `Dictionary`. For example, what if we tried to get Elizabeth Olsen from our attendance roster? Since she isn't in there, it makes no sense for us to get a value back, so we would get `nil`.

Luckily, this is a very common thing, and there is a built-in way to help deal with the potential of there not being a value. When we retrieve a value from our `Dictionary` we can provide a default value we want to get instead of `nil` if there isn't a matching `Key`. So with this example, instead of saying that they are `nil` we can just say they aren't present.

```
let isElizabethPresent = attendance["Elizabeth Olsen", default: false]
print(isElizabethPresent) // false
```

This default syntax will either give us the value that is tied to the `Key` we used, or the default value if it doesn't exist.

Tuples

A `Tuple` is a collection that lets you pass around a group of values. You can group multiple values into a single value. A big benefit of `Tuples` is that the values it contains don't need to be the same type. The values can also be named, which makes it easier to remember what data you are passing around. Tuples are, however, fixed in size, meaning you cannot add, remove, or change the `Type` of any values in them. You can change the values to a new value of the same type. You can also access data contained in a `Tuple` by either their name or their index.

```
var actor = (firstName: "Keanu", lastName: "Reeves")
// Lets use `String Concatenation` to print his name in 2 different
ways.
print("\(actor.firstName) \(actor.lastName)")
print("\(actor.0) \(actor.1)")
```

As I said before, we cannot add or remove values. In the above example, there will always be 2 strings, named first and last. However, we can change what values are stored there. Assuming we have a variable, that is.

```
actor.firstName = "Ryan"
actor.lastName = "Reynolds"
print(actor)
```

Sometimes you may just skip naming the values, that is totally fine. That just leaves you with accessing the values via index.

```
let error = (404, "Not Found")
```

When you have one of these nameless values and want to add names, you can do what is called tuple decomposition. This is where you break a tuple down into its pieces, like so:

```
let (errorNum, errorDesc) = error
// This creates two constants that represent the values of the tuple.
print(errorNum)
print(errorDesc)
```

You can do this regardless of if the tuples have named values or not

```
let (actorFirst, actorLast) = actor
print(actorFirst)
print(actorLast)
```

`Tuple` type annotation doesn't have anything crazy about it, you write the tuple but instead of putting values, you put the `Types`.

```
var fishTank: (id: Int, isDirty: Bool) = (1, false)
print("Fishtank \(fishTank.id), is dirty: \(fishTank.isDirty)")
```

When you specify a type, again you can choose to ignore the labels, but you also don't need to specify the labels if you are setting the value. This goes for both the initial assignment and later ones.

```
fishTank = (2, true)
```

Enumerations

We have a bonus topic we are going to talk about as part of our Complex Types lecture. This one isn't technically a collection, but it shares some similarities. We are going to be talking about `Enumerations` or `Enums` for short. `Enums` define a common type of related values and enables you to work with those values easily and safely.

Let's take an example of wanting to have movie genres in our application. We want to make sure that we always refer to them the exact same way, otherwise it could be confusing for the end user. We could create some properties that store them as strings and just use those right? Let's try it!

```
let comedy = "Comedy"
let adventure = "Adventure"
let action = "Action"
```

This technically works, but it could be done in a cleaner, easier way, not to mention that these values are all held so tightly together it's weird having them each be their own separate thing. This is where `Enums` come in. Let's create one.

When we create an `Enum` we are actually creating a new `Type` for Swift to recognize, so like how `String`, `Int`, `Bool`, and `Double` are all types, we are making something just like those! This does make the syntax a little different, including how we name things. So far we have named things using `lowerCamelCase` but when we create types, we create them using `UpperCamelCase`. But this is better shown than told, so let's create those movie genres.

When we are choosing the name of our `Enum` it's also important that we don't do a plural name, we want it to be singular. This can be confusing, so I want to distill it early on. So even though we are defining movie genres the name of the `Enum` will be `MovieGenre` as when we have a `MovieGenre` we only have one at a time. Finally when we declare an `Enum` we have to tell it what `Values` it can possibly be, we do this by using the `case` keyword followed by a name. This name is going to be `lowerCamelCase` like you have seen previously.

```
enum MovieGenre {
    case comedy
    case adventure
    case action
}
```

Now that we have an `Enum` let's try to use it in our code.

```
let shrekGenres: [MovieGenre] = [.comedy, .adventure, .action]
// or
let freeGuy = [MovieGenre.comedy, .adventure, .action]

print(shrekGenres)
```

When we print out the genres, we are going to get some really long text you can ignore most of that for now, just take notice the end bits where it says `.comedy`, `.adventure`, etc.

Let's try to make another example. Activities!

```
enum Activity {
     case bored
     case running
     case talking
     case coding
}
```

Associated Values

We now have this activities `Enum`, but I want there to be more information tied to it. Like for example if I'm running how far am I going, or if I'm talking what am I talking about. `Enums` have functionality for this thing exactly, called `Associated Values`. `Associated Values` allow us to tie some dynamic information to our `Enum` cases.

```
enum ValuedActivity {
     case bored // we dont always need them, we can mix and match.
     case running(miles: Double)
     case talking(topic: String)
     case coding(lines: Int, hasBugs: Bool)
}
```

Now that we defined an `Enum` with `Associated Values` when we try to refer to one of the cases, we have to pass it values that it represents.

```
let running = ValuedActivity.running(miles: 1)
let coding: ValuedActivity = .coding(lines: 65_421, hasBugs: true)
```

Raw Values

Alternatively, to having `Enums` where the information is dynamic for each value of our cases, we also have `Raw Values`. `Raw Values` are all the same type, and are declared when you declare the `Enum` meaning they are predefined. They also need to be unique, I cannot have two `Raw Values` that are the same. `Raw Values` and `Associated Values` cannot be mixed.

When we declare an `Enum` with `RawValues` we have to specify what `Type` the `Raw Value` is going to be, we do this by putting a `:` after the name followed by the `Type` we want to use. Let's do an example that represents clothing sizes.

```
enum ClothingSize: String {
     case extraSmall = "XS"
     case small = "S"
     case medium = "M"
```

```
    case large = "L"
    case extraLarge = "XL"
}
```

Now when we have an instance of our `ClothingSize` `Enum` we can access its raw value.

```
let mySize: ClothingSize = .large
print("I wear a \(mySize.rawValue) shirt!")
// or
print("I weare a \(ClothingSize.large.rawValue) shirt!")
```

A cool feature of Swift `Enums` is that certain types of `Raw Values` can be assumed, so you don't need to write them yourself.

> **Note**
>
> Typically I write the values out anyways, but you don't have to!

For example, with an `Int` `RawValue` `Type` we can put the first number on our case, and `Swift` will count up from there for the rest of our cases. Or if we use `String` as our `Raw Value` if we don't assign something different, it will use the name of our case as the `Raw Value`.

> **Error**
>
> Update raw value defaults to show how it auto generates for Strings, Ints, and Doubles Now

```
enum Planet: Int {
    case mercury = 1
    case venus
    case earth
    case mars
    case jupiter
    case saturn
    case uranus
    case neptune
    case pluto // It will always be a planet in my heart <3
}

print(Planet.pluto.rawValue) // 9
```

We can also create an `Enum` case using the `Raw Value` It's tied to like so:

```
let shirtSize = ClothingSize(rawValue: "XL") // ClothingSize.XL
let earth = Planet(rawValue: 3) // Planet.earth
```

Case Iterable

We have covered both `Collections` and `Enums` now, so let's quickly combine the two. There will be times when you need a `Collection` of your enum cases. We could always write our own, but that's tedious. Luckily, there is built-in functionality for that! We can make our `Enum` `CaseIterable` and what that does it creates a value we can access that is an `Array` of our `Enum` cases.

```
enum Drinks: CaseIterable {
    // We can declare multiple cases on the same line by comma
separating them.
    case coffee, soda, juice, water
}

print(Drinks.allCases)
print(Drinks.allCases.reversed())
```

We can do the same `CaseIterable` implementation for `Enums` with `RawValues` as well. Just make sure that your `RawValue` type comes before the `CaseIterable` and separate them with a comma!

```
enum DayOfWeek: String, CaseIterable {
    case monday = "Monday"
    case tuesday = "Tuesday"
    case wednesday = "Wednesday"
    case thursday = "Thursday"
    case friday = "Friday"
    case saturday = "Saturday"
    case sunday = "Sunday"
}

print(DayOfWeek.allCases)
```

`CaseIterable` does work for `Enums` with `AssociatedValues`, however, it requires some extra work and some stuff we haven't talked about. So we will cover it at a later date. But as for the reason, it comes down to the fact that the compiler would need to know what

values to assign to the `Associated Values`, since it doesn't, it cannot generate that information for you.